WMO Guide to Free and Open Source Software

World Meteorological Organization

Date: 2026-04-23

Version: 0.1.0

Document status: DRAFT

Document location: https://wmo-im.github.io/wmo-foss-guide/guide/wmo-foss-guide-DRAFT.html

WMO publication location: TBD

Standing Committee on Information Management and Technology (SC-IMT)[1]

Commission for Observation, Infrastructure and Information Systems (INFCOM)[2]

Copyright © 2025 World Meteorological Organization (WMO)

Table of Contents

Introduction . . . 3
- Audience . . . 3
- Scope . . . 3
- Background . . . 3
- Data policy considerations . . . 4
  - Open data as a strategic asset . . . 4
  - The importance of policy, license and legal clarity . . . 4
  - The enabling role of FOSS . . . 5
  - The way forward . . . 5
  - Supporting Links . . . 5

Guidelines . . . 7
- WMO Members . . . 7
  - Using FOSS . . . 7
  - Contributing to FOSS . . . 8
  - Managing FOSS activities . . . 9
  - Benefits of FOSS . . . 10
  - Profiling competencies . . . 10
  - Infrastructure considerations . . . 11
- WMO Activities . . . 11
  - Coordination, alignment and support . . . 12
  - Standards compliance . . . 16
  - Software review and evaluation . . . 17
  - Reference Implementations . . . 18
  - Case study: WIS2, FOSS, and agile development . . . 20

References . . . 22
- FOSS practice and governance . . . 22
- Economic impact studies . . . 22
- National and regional policies . . . 22
- Multilateral and international resources . . . 23
- Meteorological sector . . . 23

Annex A: FOSS evaluation rubric . . . 24

Annex B: Examples of FOSS in the meteorological community . . . 25

Introduction

Free and open-source software (FOSS) refers to software that is released under a license that grants users the freedom to run, study, modify, and distribute the software and its source code. Common licenses include GPL, MIT, Apache, BSD, and many others. Such software has become increasingly crucial for WMO and its Members. No longer a "niche" industry, FOSS now powers major initiatives and services worldwide, and is backed by industry support.

FOSS solutions play a pivotal role as accelerators for the implementation of Early Warnings for All, a key WMO initiative aimed at protecting every person on Earth with life-saving early warning systems by 2027. By providing accessible, customizable, and cost-effective tools, FOSS enables Members to rapidly deploy and adapt early warning systems to their specific needs and contexts. This approach is particularly crucial for developing countries (LDCs) and small island developing states (SIDS), where resource constraints often hinder the implementation of proprietary solutions.

Moreover, FOSS initiatives serve as powerful catalysts in supporting Members' efforts towards digital transformation. As National Meteorological and Hydrological Services (NMHSs) worldwide strive to modernize their operations and services, FOSS now offer a flexible and scalable foundation for innovation. They enable Members to leverage cutting-edge technologies, collaborate on development, and share best practices, thus accelerating their digital transformation journeys while optimizing resource utilization.

Audience

The document is intended for WMO Members who are involved in software development activities in their respective organizations (developers, managers/decision makers) as well as WMO Secretariat in the coordination, alignment and support of FOSS within WMO.

Scope

This document provides guidance on FOSS for WMO Members and WMO Secretariat, both in terms of using FOSS but also in the development and contribution to FOSS projects. The guidance is equally applicable to internally developed software, and to commercially sourced or supported software.

Background

FOSS development and use among WMO Members has been a longstanding activity. Typical FOSS implementations included decoders and encoder libraries of WMO BUFR and GRIB formats (e.g. ecCodes, degrib, libecbufr, etc.), as well as data dissemination services (THREDDS, GDAL, ERDDAP, etc.).

In addition, as part of the global open data/open science movement, numerous government organizations have put forth using FOSS by default; this includes use of exiting FOSS tools as well as developing of same, working in the open by default. Free software development platforms and tools (for example, GitHub) have greatly lowered the barrier for sharing software for research, development and operations for all.

As part of the development of the WMO Information System (WIS2), FOSS implementation has been a significant activity, as exemplified by a number of Reference Implementations of WIS2 Standards (such as wis2box, wis2-gdc, etc.). The development of software in the open during the pre-operational phases of WIS2 proved valuable in testing WIS2 Standards put forth in Technical Regulations while in development, promoting the “release early, release often” philosophy of agile and iterative development. While FOSS is resident in numerous WMO Member organizations, guidance and coordination becomes more vital to ensure Member services are provided with security, privacy and safety in mind for WMO Members and beyond.

Data policy considerations

While Free and Open Source Software (FOSS) is a key enabler of modern meteorological systems, its full value is realized only when combined with accessible, well-governed data. Open data and FOSS are mutually reinforcing: open data provides the raw material for analysis and forecasting, while FOSS provides the tools to discover, process, share, and operationalize that data. Together, they can e.g. support the implementation of the WMO Unified Data Policy by helping Members expand access to Earth system data and strengthen international collaboration. Together, they can e.g. support the implementation of the WMO Unified Data Policy by helping Members expand access to Earth system data, strengthen international collaboration, and contribute to broader open science practices.

Open data as a strategic asset

Open data, and especially meteorological, hydrological, atmosphere, and climate data should not be understood solely as data that is “available”. To be truly useful, data must be governed through clear policy frameworks, standards, and licensing arrangements that define how it can be accessed, shared, reused, and redistributed. Open access alone does not guarantee value. Data must also be FAIR:

- **Findable** through clear metadata and catalogues
- **Accessible** through reliable services and standard interfaces
- **Interoperable** across institutions and systems
- **Reusable** under clear legal and technical conditions

These FAIR principles are highly aligned with WMO objectives, provide a practical framework for implementing the Unified Data Policy, and help ensure that shared data can be effectively used by member states, researchers, and operational centres, thereby strengthening and supporting also open science and reproducible research.

The importance of policy, license and legal clarity

To enable meaningful data exchange, open data should be supported by a clear policy framework. Such policies help define:

- what data should be shared,
- under what conditions,

- who is responsible for stewardship,
- and how quality, security, and privacy are maintained.

Also, an often overlooked but essential aspect of open data, is licensing. Even where data is technically accessible, the absence of an explicit open data license can create uncertainty around reuse, redistribution, and related products. An open data licence helps to provide legal clarity for users and encourages broader adoption. The Open Knowledge Foundation provides a curated list of open data licences at https://opendefinition.org/licenses/.

The enabling role of FOSS

FOSS plays a central role in translating data policy into operational practice.

They can help:

- **Reducing barriers to access**: FOSS tools allow members to access and work with shared datasets without dependence on costly proprietary software, an aspect especially interesting for members with limited financial or technical resources.
- **Supporting interoperability**: Many FOSS solutions are built around open standards, APIs, and web services, making data exchange easier (see above).
- **Strengthening transparency**: FOSS workflows allow users to inspect how data are processed, improving trust, reproducibility, and scientific integrity.metadata management
- **Enabling innovation**: When open data is combined with open software, organisations can develop locally relevant applications and services, maybe also supporting start ups and local economies, while contributing improvements back to the wider community.

By relying on open standards and transparent architectures, FOSS helps ensure that data remains interoperable, portable, and reusable across institutions and Member States.

The way forward

The benefits of open data policies as the WMO Unified Data Policy depend not only on making data available, but also on ensuring that data can be effectively discovered, understood, and reused. FOSS provides the technical means to support this goal, while sound data policies and open licensing provide the governance foundation that ensures sustainable, and trusted data sharing.

Together, open data, FOSS, and open standards can help create a more collaborative, transparent, and resilient global meteorological data ecosystem.

Supporting Links

- WMO - World Meteorological Organisation
  - https://wmo.int/wmo-unified-data-policy-resolution-res1
- EU - European Union
  - https://digital-strategy.ec.europa.eu/en/policies/legislation-open-data
- ECMWF - European Centre for Medium-Range Weather Forecasts

- https://www.ecmwf.int/en/forecasts/datasets/open-data
- https://www.ecmwf.int/en/forecasts/access-forecasts/access-real-time-open-data
- https://data.ecmwf.int/

- National Policy
  - Canada - https://www.canada.ca/en/government/system/digital-government/digital-government-innovations/information-management.html

[1] https://community.wmo.int/governance/commission-membership/commission-observation-infrastructures-and-information-systems-infcom/commission-infrastructure-officers/infcom-management-group/standing-committee-information-management-and-technology-sc-imt

[2] https://community.wmo.int/governance/commission-membership/infcom

Guidelines

WMO Members

Using FOSS

Risks

- **Licence compliance risks** Open-source licences often have widely varying requirements regarding commercial use, redistribution, and code modification. Certain licences include restrictive clauses, such as mandating that derivative works be made open source or that original copyright notices be preserved in full. Violations may lead to legal disputes and intellectual property issues.
- **Security and continuity risks**: Open-source software from niche or less-established projects may pose security risks due to limited formal code audits. Additionally, the complexity of open-source software supply chains creates vulnerabilities that could be exploited to implant malicious code. These risks are amplified in meteorological applications, especially when handling sensitive observational data or real-time forecasting models.
- **Maintenance risks**: Fixes for vulnerabilities rely on community responsiveness. If an open-source project is discontinued, known vulnerabilities may remain unaddressed.
- **Dependency and stack maintenance risks**: FOSS solutions typically depend on a broader ecosystem of libraries, frameworks, and runtime components. Keeping the full software stack up to date, including across upstream dependencies, requires continuous effort. Version incompatibilities, breaking changes upstream, and end-of-life releases can create significant operational burden, particularly for small teams managing operational meteorological systems.
- **Support risks**: Most open-source software lacks official support. As a result, timely response and effective resolution of critical operational system failures cannot be guaranteed, potentially leading to interruptions in meteorological operations and data loss.

Hidden costs

- **Technical adaptation and integration costs**: Human resources must be allocated for software customization and adaptation, system interface integration, and compatibility testing to ensure the software aligns with existing systems.
- **Long-term maintenance and version management costs**: Dedicated personnel are needed to track version updates, conduct testing, and manage migrations. If a project is discontinued, taking over its maintenance internally entails high technical challenges and added operational complexity.
- **Talent acquisition and training costs**: Recruiting skilled professionals can be expensive, and ongoing in-house training is required to maintain the necessary expertise.
- **Compliance audit and management costs**: It is necessary to maintain an inventory of all open-source components and perform regular compliance audits. Large meteorological organizations may also need to develop dedicated management platforms, with costs increasing as the number of components grows.

Contributing to FOSS

National policies

The adoption and development of FOSS within NMHSs can be significantly strengthened through supportive national-level policies. Several WMO Members have already introduced digital government strategies that encourage the use of open standards, open data and open-source software as default options within public administrations.[1] Where such frameworks exist, NMHSs can leverage them to increase efficiency, reduce vendor lock-in and ensure long-term sustainability of operational systems.

National FOSS-related policies may include:

- "Open by default" procurement rules requiring public institutions to consider FOSS alternatives alongside proprietary solutions when acquiring software. National examples include the United States Federal Source Code Policy,[2] the Government of Canada Open First approach,[3] Australia's Digital Service Standard (Criterion 8: Make source code open),[4] New Zealand's NZGOAL-SE framework,[5] India's Policy on Adoption of Open Source Software,[6] and Japan's Open Source Promotion Report.[7] In some countries, like Romania, when you purchase "on demand" software development services, the source code should be transferred to the customer along with all the IPR associated with the software. Making the source code available under an open-source licence is highly recommended.
- Guidelines for software development funded from public budget, encouraging publication of code under approved open-source licences. The "Public Money? Public Code!" campaign, supported by over 200 civil society organizations, advocates that software developed with public funds should be released under open-source licences.[8]
- Policies promoting interoperability and open standards, which naturally align with FOSS implementations within WIS2 and other WMO technical frameworks. In the European Union, the Interoperable Europe Act[9] (in force from April 2024) requires public sector bodies to conduct interoperability assessments and share open source solutions for cross-border digital services.
- Capacity-building programmes that enable staff to acquire skills for maintaining and developing open-source tools, reducing long-term operational dependency on external vendors.
- Security and lifecycle requirements to ensure that open-source components used in critical systems follow maintenance, patching, auditing and version management rules.

Where no national regulations exist, NMHSs can still develop institution-level policies or internal guidelines to support responsible FOSS adoption:

- Establish internal rules for licensing, contribution approval and repository management.
- Define procedures for evaluating FOSS components (governance, community health, maintenance model).
- Incorporate sustainability planning (including contribution back to upstream projects when fixes or new features are developed internally).
- Promote transparency by adopting open repositories for research software and operational tools.

- Events/hackathons (e.g. OGC/OSGeo/ASF Joint Sprints) TODO Vasile
  - By product: connection/collab
- Regulations / risk / constraints / considerations
  - Features, maintenance, project

Managing FOSS activities

In some cases, Members developing their own FOSS in support of WMO obligations and activities may wish to make the software freely available to the public. The current FOSS landscape and infrastructure provides a strong support mechanism for doing FOSS development in the open, using services such as GitHub, GitLab, Codeberg and numerous other platforms.

In the same spirit as contributing to FOSS, where no national regulations exist, NMHSs can still develop institution-level policies or guidelines in support of making FOSS available to the public. The following are (some) key aspects to consider:

- Establish an "architecture of participation":
  - Who will be responsible for development, release management, and accepting external contributions?
  - How will commit access be governed and managed? Does the project need a project steering committee or group of stakeholders to help drive the project?
  - How will external collaboration be addressed? For example, will the project have a bug or issue tracker which can be used to identify new feature development, defects and/or bugs. Will there be a dedicated discussion forum or mailing list for wider discussion?
  - Make clear how collaboration will work to the public (for example, by providing a `CONTRIBUTING.md` document for GitHub)
  - Make clear how security issues should be addressed (mitigation/release policies/timelines, contact points, etc.)
  - What is the release schedule for new versions?
- Choose a licence that the software will be made available with (such as the Apache License, Version 2.0, MIT, etc.). It is strongly advised to consider and consult organization policies to ensure proper licensing and understanding risk, responsibility and liability
- Choose the mechanism by which the software will be made available (GitHub, GitLab, Codeberg, organizational website, etc.)
- Ensure that a `README` is available with the code stating a clear mission and purpose
- Identify any supported WMO Technical Regulations
- Ensure that the project has proper documentation for users (downloading, installing, configuring, running), as well as additional documentation for developer or other audience types (administrators, etc.)
- Ensure the code is versioned (for example using Git as a version control system)

Benefits of FOSS

The benefits of FOSS can be summarized across three dimensions: autonomy and control, cost optimization, and ecosystem flexibility.

- **Autonomy and control**: The code is open and licences are flexible, allowing meteorological organizations to perform customization as needed, directly modifying the code to add or remove functional modules to suit unique operational scenarios (e.g., regional weather forecasting adjustments). This level of access also allows them to independently identify and fix vulnerabilities, optimize performance for large meteorological data workloads, and control the technical roadmap of core systems. Certain permissive licences (e.g., MIT License, Apache License 2.0) even permit using modified code as closed source in commercial products, balancing customization needs with operational requirements.
- **Cost advantages**: No software licence fees or subscription fees are required, making FOSS suitable for national meteorological services as well as deployments in regional forecasting centres. The availability of source code and community documentation reduces costs during adaptation and integration. Meteorological organizations can also choose free community support or affordable third-party services instead of expensive vendor support packages. A 2021 study commissioned by the European Commission estimated that EUR 1 billion of investment in open source generated EUR 65–95 billion in economic impact, characterizing FOSS as a global public good.[10] A World Bank study of institutional investment in open-source geospatial software found a conservatively estimated 200% return on investment over seven years.[11]
- **Avoiding vendor lock-in**: Free from proprietary interfaces and data format dependencies, enabling free choice of deployment environments (e.g., private or public clouds). Even if the original project is discontinued, maintenance can be taken over independently or by a third party. Moreover, multiple service providers often support the same open-source software, strengthening the user's bargaining position.
- **Cross-platform portability**: Most open-source tools run on multiple operating systems (e.g., Windows and Linux). Adherence to common technical standards (e.g., WMO GRIB data formats) lowers system integration difficulty. Modular design facilitates containerized deployment and cross-cluster migration, adapting to architectural upgrades in meteorological data centres.
- **Technical capacity development**: Community involvement within FOSS projects can foster career growth, professional recognition, networking and collaboration. In turn this can lead to increased techical capacity within the Members NMHSs and other organisations.

Profiling competencies

Members wishing to use, participate and/or manage FOSS activities should be familiar with basic FOSS principles, and assess their level of participation accordingly. Understanding the architecture of participation of FOSS varies among projects. In addition to domain and software development competencies, staff require the ability to analyze a given FOSS project and/or community, understanding how development, contributions and social and political infrastructure operate. Staff should also be supported to participate in FOSS as part of their ongoing learning and development. This can include attending conferences and/or code sprints and hackathons. The degree to which these skills are developed depend on the level of FOSS investment by a given Member organization.

Infrastructure considerations

FOSS development and distribution is supported by a rich ecosystem of freely available hosting platforms and services. Understanding this landscape helps Members make informed choices when publishing software, consuming FOSS components, or participating in collaborative development.

Source code hosting and collaboration

Platforms such as GitHub, GitLab, and Codeberg provide the core infrastructure for hosting source code, tracking issues, managing contributions via pull or merge requests, and publishing project documentation. Many of these platforms also offer free tiers for public open-source projects. Members should consider platform accessibility (e.g. connectivity constraints in some regions), governance (commercial vs. community-run), and data sovereignty when making a choice. Some platforms, notably GitLab (Community Edition), Gitea, and Forgejo, can also be self-hosted, which may be preferable where organizational policy or national regulations require software infrastructure to remain under institutional control.

Continuous integration and delivery (CI/CD)

Most major hosting platforms include integrated CI/CD tooling (e.g. GitHub Actions, GitLab CI) that enables automated testing, building, and release workflows. These services are typically free for public repositories and significantly reduce the overhead of maintaining software quality and producing releases. Where platforms are self-hosted, organizations must also provision and maintain the runner infrastructure — the compute resources on which CI/CD jobs execute. This may be on-premises hardware, virtual machines, or cloud instances, and represents an additional operational consideration when choosing a self-hosted deployment model.

Package and artefact registries

FOSS components are distributed through a variety of public registries depending on the language and runtime environment — for example, PyPI (Python), npm (JavaScript), Docker Hub and GitHub Container Registry (container images), and conda-forge (scientific computing). Members consuming FOSS should be aware of the provenance and integrity guarantees offered by each registry, and consider mirroring critical dependencies where connectivity is a concern. Registry licensing terms can also change: for example, Anaconda Inc. revised their Terms of Service in 2020 to require a paid licence for use of their `defaults` channel by organizations with more than 200 employees, while the community-maintained conda-forge channel remains freely available. This illustrates the importance of reviewing registry terms alongside software licences.

Community infrastructure

Beyond code hosting, FOSS projects often rely on additional community infrastructure including mailing lists, discussion forums, chat platforms (e.g. Matrix, Slack, Discourse), and documentation sites. When evaluating or adopting a FOSS project, the health and accessibility of these channels is a useful indicator of community activity and support.

WMO Activities

Coordination, alignment and support

As FOSS has become increasingly embedded in WMO programmes, projects and Member operations, the need for structured coordination at the organizational level has grown accordingly. Historically, software developed, or selected, for use in different projects and programmes has largely been managed on an ad-hoc basis. Independent experts, contractors, and implementing authorities / agencies have typically made independent choices without a common framework for the evaluation, governance, and life-cycle management of FOSS.

Whilst enabling rapid development and innovation, this approach has led to fragmentation and duplication of effort, and in some cases, reliance on unmaintained or insecure software that puts Members services at risk.

The WMO Secretariat, as the body supporting the work of its Members and constituent bodies, has a central role in coordinating FOSS activities across WMO programmes and projects and in supporting Members in the adoption of FOSS. This coordination function should ensure that FOSS initiatives adopted through WMO projects and programmes are aligned with the WMO's regulatory framework and that software used in support of WMO obligations meets appropriate standards of quality and security. Equally important, this coordination should ensure that Members, particularly those with more limited technical capacities, are not left behind.

Governance and organizational functions

Effective FOSS coordination with the WMO requires dedicated governance functions. The following functions are identified as essential:

- **Software evaluation and review**: Providing a consistent process for assessing FOSS used in WMO programmes and projects, applying criteria such as those described in the FOSS evaluation rubric (see FOSS evaluation rubric). This includes evaluating licence compatibility, security posture, project and community health, openness of governance, and technical fitness for purpose.
- **Lifecycle oversight**: Tracking the status of FOSS components used across WMO programmes, from initial adoption through operational use to end-of-life, ensuring that migration plans are in place when software is no longer maintained.
- **Standards alignment**: Ensuring that FOSS developed or endorsed within WMO implements relevant Technical Regulations and open standards correctly and consistently.
- **Guidance and advisory support**: Providing practical guidance to Expert Teams, Task Teams, Communities of Practice, and other implementors on FOSS best practices, including licensing, contribution workflows, governance models and release management.

These functions may be carried out by existing WMO bodies, such as the Standing Committees, Expert Teams, and Task Teams. However, these bodies often have a narrow focus on a particular activity or area, are not typically involved in the projects implementing or using FOSS, and often lack the specialist knowledge on FOSS. Alternatively, a dedicated software review function, such as through an Open Source Program Office (OSPO), could be established. Regardless of the organizational form, there is a clear requirement for software governance to be systematic rather than ad-hoc, and that clear, objective criteria exist for evaluating and endorsing FOSS within WMO.

It is important to note that WMO should not seek to develop its own registry of open-source software.

Existing registries and ecosystems (such as those maintained by OSGeo, the Digital Public Goods Alliance, and language-specific package repositories) already serve this purpose. WMO's coordination role should focus on evaluating and recommending software from within these existing ecosystems, rather than duplicating their efforts.

Roles and responsibilities

Coordination of FOSS activities within the WMO involves a range of actors:

- **WMO Secretariat**: Facilitates coordination across programmes and projects, supports the development and maintenance of FOSS guidance, and manages relationships with external FOSS organizations. The Secretariat also provides continuity and institutional memory for FOSS initiatives that span multiple constituent body sessions.
- **Expert Teams and Task Teams**: Provide domain-specific expertise in evaluating and developing FOSS relevant to their areas of responsibility. Expert Teams and Task Teams may identify candidate software for endorsement or develop Reference Implementations of WMO standards.
- **Communities of Practice**: Serve as collaborative spaces where Members, developers and stakeholders work together on shared FOSS initiatives. Communities of Practice can play an important role in maintaining and evolving software that supports WMO standards, as demonstrated by the development of Reference Implementations within the WIS2 programme and the OpenCDMS project and initiative.
- **Members**: Contribute expertise, code, testing and operational feedback. Members with mature FOSS programmes can serve as mentors and collaborators for those building capacity.

Whilst "Communities of Practice" have been listed above, these do not exist formally within the WMO's framework and constituent bodies, instead operating on an informal basis. Strengthening and formally recognizing these communities would have significant benefit.

FOSS project life cycle

FOSS projects and software typically progress through several stages:

- **Concept and prototyping**: An initial requirement is identified and ideas are explored. These may often be part of the development of a new standard, such as the WIS2.0 standards, or in response to an operational need. At this stage, the software may be experimental and maintained by a small number of developers.
- **Development and pilot use**: Following initial prototyping, the software gains maturity and may have additional contributors. The software is tested in pilot environments. Parallel to this, the standards that the software is implementing may also gain maturity and undergo further changes.
- **Operational use**: Software, and standards, are ready for production use in operational environments. There will be established governance processes for the FOSS together with comprehensive documentation and contribution guides. The contributor base will have grown

and be sustainable.

- **End-of-life and migration**: When software is no longer actively maintained or has been superseded, clear guidance should exist for migrating to alternative solutions, including data migration. End-of-life planning is particularly important for software that Members depend on for operational services.

The WMO can support this lifecycle by providing evaluation criteria at each stage, recognizing software that meets established thresholds of maturity, and facilitating transitions when software reaches end-of-life. Established frameworks such as the OSGeo incubator and graduation process[12] provide a useful model for structuring such a lifecycle within the WMO.

FOSS sustainability and risk management

Sustainability, and long-term maintenance and support, are amongst the most significant challenges in FOSS adoption. Software that is relied upon by Members, especially for operational services, must be maintained over the long-term, with timely security updates and patches, backwards compatibility and responsive support. Key risks to be managed include:

- **Bus / retirement factor**: Over-reliance on a single, or small team, for critical software. When supporting FOSS development and adoption, the WMO should encourage a broad contributor base, shared governance, and pooling of resources for the support and development of critical software.
- **Lack of long-term support and maintenance**: Not all FOSS projects have a sustained commitment to long-term support. Projects may cease active development without formal end-of-life notice, leaving Members dependent on unmaintained software with unaddressed vulnerabilities and no migration path. Unlike proprietary software, where support obligations are contractually defined, FOSS maintenance relies on community goodwill, institutional backing, or commercial arrangements — none of which are guaranteed. Members should assess the long-term support model of any FOSS solution prior to operational adoption and have contingency plans in place should maintenance cease.
- **Security vulnerabilities**: Software components must be subject to regular security reviews, including vulnerability testing, with clear disclosure and patching policies. Adequate resources must be in place to ensure any identified vulnerabilities are addressed in a timely fashion.
- **Dependency risk**: Software often depends on a chain of upstream libraries and components. Understanding and monitoring these dependencies is essential for assessing overall project health. This includes changes to upstream licensing conditions.
- **Funding and institutional support**: Whilst FOSS may be free of licensing costs there are still costs associated with the development, maintenance, implementation and operation of FOSS software. These costs are often hidden or neglected but should be taken into account. Capacity development costs — including staff training, upskilling, and the ongoing effort required to build and retain the expertise needed to evaluate, deploy and maintain FOSS — represent a significant and frequently underestimated component of these costs, particularly for Members with limited technical resources.

The WMO has a role in supporting FOSS initiatives and managing risks by promoting and supporting investment in the development and maintenance of FOSS software and in community management. This support may be realized through sources such as the WMO Voluntary

Cooperation Programme (VCP), extra-budgetary resources and projects, and through broader stakeholder and donor engagement.

The WMO's coordination role should include monitoring the health of FOSS that is critical to WMO programmes and alerting Members to emerging risks, such as previously promoted projects that show declining activity, unpatched vulnerabilities or loss of key maintainers.

Alignment with the WMO ecosystem

In the WMO context, FOSS initiatives should be aligned with the broader set of WMO activities in supporting Members. This includes:

- **Technical Regulations and standards**: Co-development of Reference Implementations alongside the development and implementation of new regulations and standards; FOSS developed or endorsed should conform with and / or implement WMO standards. The WMO should also promote alignment and interoperability with open standards, such as those developed by the Open Geospatial Consortium (OGC).
- **Cross-programme and cross-project coordination**: FOSS components are often relevant to multiple WMO programmes. Coordination can help avoid duplication, identify opportunities for shared development and ensure that Members benefit from a coherent set of tools rather than competing or overlapping solutions.

Supporting Members

A key function of WMO coordination is ensuring that all Members can benefit from FOSS, regardless of their technical capacity, and making sure that no Member is left behind. This includes:

- **Capacity building**: Providing training, documentation and technical support to help Members evaluate, deploy and maintain FOSS solutions. This is particularly important for developing countries and small island developing states, where resource constraints may limit the ability to engage with FOSS communities directly.
- **Knowledge transfer**: Facilitating the exchange of experience and expertise between Members, for example through workshops, code sprints and hackathons organized alongside WMO events or in collaboration with partner organizations.
- **Visibility and discovery**: Helping Members identify FOSS that is relevant to their needs by curating recommendations and maintaining awareness of available tools within the WMO community, drawing on existing registries and partner ecosystems.

External partnerships

The WMO does not operate in isolation in the FOSS landscape. Established organizations such as the Open Source Geospatial Foundation (OSGeo), the Digital Public Goods Alliance (DPGA), the Apache Software Foundation and others have developed mature frameworks for FOSS governance, incubation and community management. WMO should seek to leverage these existing frameworks rather than developing parallel structures.

Potential areas for collaboration include:

- **Formal partnerships:** Memoranda of Understanding (MoUs) with organizations such as OSGeo

can provide a structured basis for collaboration on project governance, incubation and quality assurance.

- **Registration as Digital Public Goods:** Key WMO FOSS implementations may qualify for registration with the DPGA, increasing their visibility and credibility within the broader international development community. The DPG Standard provides a structured quality framework covering licence compliance, documentation, privacy, and standards adherence.
- **Multilateral development partners:** The UNDP and ITU, through their joint Open Source Ecosystem Enabler (OSEE) initiative, provide practical support to developing countries in establishing national open source ecosystems and Open Source Programme Offices (OSPOs). The World Bank maintains its own open source programme and has documented significant returns on institutional FOSS investment in geospatial and environmental data domains directly analogous to NMHS operations.[13]
- **Joint events:** Code sprints, hackathons and workshops organized in collaboration with partner organizations (such as OGC/OSGeo/ASF joint sprints or FOSS4G conferences) provide valuable opportunities for capacity building, community engagement and collaborative development.
- **Meteorological sector examples:** The European Centre for Medium-Range Weather Forecasts (ECMWF) runs an annual open source innovation programme under the Copernicus Climate Change Service (Code for Earth),[14] demonstrating how a major operational meteorological institution can embed FOSS practice as a core activity.
- **Regional digital innovation hubs**: Regional digital innovation hubs, established in collaboration with Members, national and international development agencies, and regional organizations, can play an important role in the development and implementation of FOSS solutions, with a focus on meeting the needs of Members within a region. These could be co-located, or hosted, by existing regional centres.

Standards compliance

FOSS can also play a significant role as an indicator of the feasibility of Technical Regulation implementation. As part of the standards development phase, developing the associated FOSS implementation(s) aids in determining "fit for purpose" of the specification. In addition, FOSS implementation during standards development allows for an open review and assessment on how a standard is implemented (supported network protocols, avaiability of tooling/libraries for integration, as well as speed of implementation (if a FOSS implementation during the development of a standard takes a considerable level of effort, why? Is the complexity acceptable for the scope of the standard? Determining these aspects earlier can help improve and adjust the standard while in development (as opposed to formal amendment processes once approved by WMO). The agile and parallel development of WIS2 standards and FOSS is a key example in this context (see Case study: WIS2, FOSS, and agile development).

Another indicator is the number of implementations. For example, the Open Geospatial Consortium (OGC) mandates that a proposed standard requires at least 3 implementations for consideration by its Technical Committee. This provides "proof" to the Technical Committee that the standard can be implemented, and has investment from various projects as early adopters.

In summary, while FOSS is not in scope for being identified as part of a Technical Regulation, it is a strong indicator of the the maturity, applicability and sustainability of a given standard.

Implementing FOSS during standards development provides safety in investment, iterative improvements, and confidence for widespread and sustainable adoption and implementation by Members.

Software review and evaluation

The identification and selection of software for WMO-related activities and Member implementations should follow a structured, transparent and repeatable assessment process. This applies equally to FOSS and proprietary solutions and aims to reduce technical, operational and organizational risks while increasing confidence in long-term sustainability, interoperability and compliance with WMO frameworks. Before adopting, recommending or endorsing a software solution, Members and WMO bodies are encouraged to perform a lightweight but systematic project assessment covering at least the following dimensions:

- Purpose and scope alignment. The software should have a clearly defined purpose and scope, aligned with the intended operational, research or regulatory use case. Its role within the overall system architecture (e.g. data production, exchange, discovery, processing or visualization) should be explicit and overlaps or dependencies with existing solutions should be identified.
- Standards and regulatory compliance. The software should demonstrably support relevant WMO Technical Regulations, standards and recommended practices, as applicable (e.g. WIS2, BUFR, GRIB, NetCDF, WMO Core Metadata Profile). Where full compliance is not yet achieved, gaps, limitations and mitigation measures should be clearly documented.
- Licensing and policy compatibility. Licensing terms must be clearly stated and compatible with national regulations, WMO data policies and operational constraints. For FOSS, licences should be OSI-approved and allow redistribution, modification and operational use without legal ambiguity.
- Project maturity and sustainability. The assessment should consider the maturity of the project, including release history, roadmap availability, versioning practices and evidence of operational use. Indicators such as the number of active contributors, governance transparency, institutional backing and the “bus factor” provide insight into long-term sustainability and associated risks.
- Security and risk management. The project should have documented security practices, including vulnerability reporting mechanisms, patching policies and incident response procedures. Known risks, external dependencies and lifecycle considerations (including end-of-life or migration paths) should be evaluated early in the selection process.
- Technical quality and operational fitness. Software quality should be assessed based on documentation availability, test coverage, automation (e.g. CI/CD), performance at NMHS-relevant data volumes, and ease of deployment, operation and maintenance. Evidence of production use within the WMO community or by peer organizations is a strong positive indicator.
- Organizational readiness and total cost of ownership (TCO). Adopting organizations should evaluate internal capacity, including staff skills, support models, infrastructure requirements and TCO. This includes training needs, long-term maintenance effort and the ability to contribute fixes or enhancements upstream where appropriate.

To support early-stage identification and comparison, Members and WMO activities may also consult community-maintained software catalogues, such as the FOSS4G Observatory (https://project.oss4geo.org/). These platforms provide structured metadata for a wide range of open-source software projects, including functional scope, licensing, governance characteristics, development activity and dependency relationships. Such catalogues can assist with preliminary screening and ecosystem-level understanding, but the information they provide should be treated as indicative and complementary.

For consistency and transparency, WMO encourages the use of a common evaluation checklist or rubric, such as the one provided in Annex A, and promotes rolling reviews of selected software to account for evolving requirements, standards and ecosystem changes. This assessment process does not seek to formally "approve" specific software for inclusion in WMO Technical Regulations, but rather to provide confidence-based guidance and shared criteria to support informed decision-making and harmonized adoption across the WMO ecosystem.

Reference Implementations

Reference Implementations (RIs) are key resources for the precise implementation of a given standard or requirement set. Benefits of RIs include (but are not limited to):

- validating the functionality and capability of a given standard (for example, for data encodings or interfaces)
- providing a publicly available FOSS resource that is definitive and can be studied to better understand and/or implement a standard
- providing an implementation that may be used by Members as desired

RIs developed alongside a given specification are especially useful for agile, rapid and iterative feedback during the specification definition phase, leading to the approval of Technical Regulations with confidence. It is therefore strongly recommended to develop RIs as part of WMO Technical Regulation development to ensure fit for purpose and functionality. Citing a RI as part of a WMO Technical Regulation should not be required, but allowed if applicable.

NOTE	They key purpose of an RI is functionality coverage. They need not be suitable for mission critical or production environments.

Compliance (data exchange)

- **Standards alignment**: The software adheres to WMO technical standards and common industry specifications, unifying data exchange interfaces and formats.
- **Data protection**: It complies with international conventions and national data protection regulations by implementing data desensitization, encrypted data transmission, and secure storage of sensitive meteorological data.
- **Cross-border data security**: It meets data cross-border transfer requirements in operational regions by establishing a full lifecycle data-security management mechanism. This mechanism covers data collection, processing, sharing, and archiving in line with WMO data policies.

Software evaluation

- **Core Evaluation Checklist**:
  - *Technical Quality*: Functionality completeness and accuracy for meteorological use cases, performance when processing massive meteorological datasets (e.g., real-time observational data, forecast model outputs), reliability under peak load conditions, security, maintainability, and compatibility with existing meteorological systems.
  - *Compliance*: Licence is OSI-approved with no licence conflicts, and complies with WMO data policies and relevant regulations.
  - *Community Sustainability*: Recent community activity (within the last 6–12 months), stability of the maintenance team, regularity of version releases, and maturity of the ecosystem.
  - *Cost-effectiveness*: Controllable costs across the software lifecycle (deployment, operation, maintenance, training, migration, and decommissioning).
  - *Business Fit*: Aligns with existing meteorological operational processes, good usability for forecasters and data analysts, and support for future expansion needs.
- **Confidence Assurance:**
  - Adopt standardized evaluation processes and quantifiable metrics to ensure objectivity and repeatability of results.
  - Validate core functionality through a Proof-of-Concept (PoC) approach, incorporating feedback collected from real-world operational scenarios.
  - Conduct multi-dimensional verification, including code audits, security vulnerability scans, and in-depth open-source community research, to confirm the reliability of the software.

Readiness

Completed drafting of requirements specification documents, and form a professional evaluation team (including technical, business, legal, and operations personnel).

Set up a simulated meteorological operations testing environment, and prepare test data (including extreme weather scenario data) and necessary evaluation tools.

Define clear priorities for software requirements and establish final decision criteria, such as approval for full adoption, conditional adoption, or rejection. Document the rationale for all decisions for audit purposes.

Bus/retirement factor

Business Fit and Retirement Considerations

- Business Fit
  - Offers seamless integration with existing meteorological operational processes, minimizing the need for process adjustments.
  - Supports custom development and feature expansion to meet evolving needs (such as business scaling and new scenario integration).
  - Aligns with users' habits in meteorological operations by providing a user-friendly interface

and flexible configuration options.

- Retirement Planning
  - Establish a clear data archiving plan to ensure secure storage and traceability of historical meteorological data after the software is decommissioned.
  - Proactively assess the costs of migrating to new systems, and establish a smooth migration path to ensure business continuity.
  - Define clear responsibilities and standardized procedures for software retirement to mitigate potential risks (e.g., data leakage, system conflicts, and service disruptions).

Case study: WIS2, FOSS, and agile development

The WMO Information System 2.0 (WIS 2.0) is the framework for WMO data sharing in the 21st century for all WMO domains and disciplines. It supports the WMO Unified Data policy and the Global Basic Observing Network (GBON) and makes international, regional, and national data sharing simple, effective, and inexpensive. The idea that no Member should be left behind and the objective of lowering the barrier to adoption has been at the core of WIS 2.0 development. These objectives inspire the principles underpinning the WIS 2.0 technical framework, such as adopting open standards and Web technologies to facilitate sharing of increasing variety and volume of real-time data.[15]

The WMO Executive Council, through Resolution 34 (EC-76) - Implementation plan update of the WMO Information System 2.0, endorsed the WMO Information System 2.0 (WIS2) implementation plan. The Resolution also recognized the importance of establishing a pilot phase to develop the WIS2 infrastructure and begin testing it, in order to be ready for a pre-operational phase in 2024, and then for the transition starting in 2025.

The pilot phase was completed at the end of 2023, with several Members collaborating in building the WIS2 infrastructure. Each Member had a different role in the WIS2 framework and implemented a specific component. Starting in January 2024, the implementation of WIS2 entered the pre-operational phase in preparation for transitioning to WIS2. On 01 January 2025, WIS2 became operational.[16]

WIS2 is comprised of a number of key standards, including (but are not limited to):

- Internet and messaging protocols for data and metadata transmission (HTTP, MQTT)
- metadata encodings for dataset descriptions (WMO Core Metadata Profile) and notifications (WIS2 Notification Message)
- topic structures for real-time notifications (WIS2 Topic Hierarchy)
- interfaces and APIs for Global Services (Global Discovery Catalogue)

Throughout the WIS2 Implementation timeline, the architecture, standards definitions as well as key software components were developed, deployed and tested in parallel, in an agile manner. These software include (but are not limited to):

Software	WIS2 component(s)
wis2box	WIS2 Node

Software	WIS2 component(s)
wis2-gdc	Global Discovery Catalogue
csv2bufr	CSV to BUFR encoding tool for station data
wis2downloader	Python package for downloading real-time data from WIS2
pywis-pubsub	Python package providing notification message publishing, subscription and data download capability in WIS2

Driven by collaboration, the development and testing of FOSS significantly helped with early assessment of the WIS2 standards in development. As a result, WIS2 standards were being tested in an agile environment and rapid feedback was provided that helped refine, adjust and improve the standards early and often.

WIS2 FOSS implementations have resulted in a more stable and robust programme delivery with significantly reduced risk to Members. In addition, some tools have emerged as Reference Implementations. WIS2 Technical Regulations were approved with confidence as a result of the agile development of open standards and FOSS.

[1] A global database of government open source software policies is maintained by the Center for Strategic and International Studies (CSIS): https://www.csis.org/programs/strategic-technologies-program/resources/government-open-source-software-policies. Country-level intelligence reports for 60+ countries are also available from the EU Open Source Observatory (OSOR): https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/open-source-software-country-intelligence

[2] OMB Memorandum M-16-21, August 2016: https://obamawhitehouse.archives.gov/sites/default/files/omb/memoranda/2016/m_16_21.pdf

[3] https://www.canada.ca/en/government/system/digital-government/digital-government-innovations/open-source-software/open-first-whitepaper.html

[4] https://www.dta.gov.au/help-and-advice/digital-service-standard/digital-service-standard-criteria/8-make-source-code-open

[5] https://www.data.govt.nz/manage-data/policies/nzgoal/nzgoal-se-guidance-note-1/

[6] Ministry of Electronics and Information Technology (MeitY), 2015: https://www.meity.gov.in/static/uploads/2024/02/policy_on_adoption_of_oss.pdf

[7] Information-technology Promotion Agency (IPA), 2024: https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/news/japans-2024-open-source-promotion-report

[8] https://fsfe.org/activities/publiccode/index.en.html

[9] Regulation (EU) 2024/903: https://interoperable-europe.ec.europa.eu/interoperable-europe/interoperable-europe-act

[10] Blind, K. et al.: The Impact of Open Source Software and Hardware on Technological Independence, Competitiveness and Innovation in the EU Economy, European Commission, 2021: https://digital-strategy.ec.europa.eu/en/library/study-about-impact-open-source-software-and-hardware-technological-independence-competitiveness-and

[11] GFDRR/World Bank: OpenDRI and GeoNode — A Case Study for Institutional Investment in Open Source: https://blogs.worldbank.org/en/opendata/leveraging-open-source-public-institution-new-analysis-reveals-significant-returns-investment-open

[12] https://www.osgeo.org/resources/project-graduation-checklist/

[13] GFDRR/World Bank: OpenDRI and GeoNode — A Case Study for Institutional Investment in Open Source: https://blogs.worldbank.org/en/opendata/leveraging-open-source-public-institution-new-analysis-reveals-significant-returns-investment-open

[14] https://climate.copernicus.eu/code-earth-2025-advancing-energy-system-resilience-climate-data

[15] https://wmo.int/wis-20

[16] https://wmo-im.github.io/wis2-transition-guide/transition-guide/wis2-transition-guide-APPROVED.html#_1_introduction

References

FOSS practice and governance

- Fogel, Karl: Producing Open Source Software: How to Run a Successful Free Software Project (2024) [1]
- European Commission / Open Source Observatory: OSOR Handbook on Open Source Software in Public Administration (2026) [2]
- Public Digital: Open Source in Government (2021) [3]
- Free Software Foundation Europe: Public Money? Public Code! [4]

Economic impact studies

- Blind, K. et al.: The Impact of Open Source Software and Hardware on Technological Independence, Competitiveness and Innovation in the EU Economy, European Commission (2021) [5]
- GFDRR/World Bank: OpenDRI and GeoNode — A Case Study for Institutional Investment in Open Source [6]
- Linux Foundation: Active Open Source Contribution Delivers 2–5x ROI (2025) [7]

National and regional policies

- European Commission: Open Source Software Strategy 2020–2023 — Think Open (2020) [8]
- Regulation (EU) 2024/903: Interoperable Europe Act [9]
- Regulation (EU) 2024/2847: Cyber Resilience Act — open source implications [10]
- United States Office of Management and Budget: Federal Source Code Policy (OMB M-16-21, 2016) [11]
- Government of Canada: Open First Whitepaper (2020) [12]
- Australian Government Digital Transformation Agency: Digital Service Standard — Criterion 8: Make source code open [13]
- New Zealand Government: NZGOAL Software Extension (NZGOAL-SE) [14]
- Ministry of Electronics and Information Technology (MeitY), India: Policy on Adoption of Open Source Software for Government of India (2015) [15]
- Information-technology Promotion Agency (IPA), Japan: 2024 Open Source Promotion Report [16]
- Government of Brazil: Portal do Software Público Brasileiro [17]
- African Union Commission: Digital Transformation Strategy for Africa 2020–2030 [18]
- Center for Strategic and International Studies (CSIS): Government Open Source Software Policies database [19]

Multilateral and international resources

- Digital Public Goods Alliance: Digital Public Goods Standard [20]
- UNDP / ITU: Open Source Ecosystem Enabler (OSEE) [21]
- UNDP Digital Standards — Standard 9: Default to Open [22]
- World Bank: Open Source for Global Public Goods (2020) [23]
- EU Open Source Observatory: Open Source Software Country Intelligence (60+ countries) [24]

Meteorological sector

- ECMWF / Copernicus Climate Change Service: Code for Earth [25]
- WMO: WIS2 Transition Guide [26]

[1] https://producingoss.com

[2] https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/news/updated-osor-handbook-published

[3] https://assets.public.digital/Open_Source_Report.pdf

[4] https://fsfe.org/activities/publiccode/index.en.html

[5] https://digital-strategy.ec.europa.eu/en/library/study-about-impact-open-source-software-and-hardware-technological-independence-competitiveness-and

[6] https://blogs.worldbank.org/en/opendata/leveraging-open-source-public-institution-new-analysis-reveals-significant-returns-investment-open

[7] https://www.linuxfoundation.org/research/open-source-roi

[8] https://commission.europa.eu/about/departments-and-executive-agencies/digital-services/open-source-software-strategy_en

[9] https://interoperable-europe.ec.europa.eu/interoperable-europe/interoperable-europe-act

[10] https://digital-strategy.ec.europa.eu/en/policies/cra-open-source

[11] https://obamawhitehouse.archives.gov/sites/default/files/omb/memoranda/2016/m_16_21.pdf

[12] https://www.canada.ca/en/government/system/digital-government/digital-government-innovations/open-source-software/open-first-whitepaper.html

[13] https://www.dta.gov.au/help-and-advice/digital-service-standard/digital-service-standard-criteria/8-make-source-code-open

[14] https://www.data.govt.nz/manage-data/policies/nzgoal/nzgoal-se-guidance-note-1/

[15] https://www.meity.gov.in/static/uploads/2024/02/policy_on_adoption_of_oss.pdf

[16] https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/news/japans-2024-open-source-promotion-report

[17] https://github.com/spbgovbr

[18] https://au.int/sites/default/files/documents/38507-doc-dts-english.pdf

[19] https://www.csis.org/programs/strategic-technologies-program/resources/government-open-source-software-policies

[20] https://new.digitalpublicgoods.net/standard

[21] https://www.undp.org/digital/osee

[22] https://www.undp.org/digital/standards/9-default-to-Open

[23] https://documents.worldbank.org/en/publication/documents-reports/documentdetail/672901582561140400/open-source-for-global-public-goods

[24] https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/open-source-software-country-intelligence

[25] https://climate.copernicus.eu/code-earth-2025-advancing-energy-system-resilience-climate-data

[26] https://wmo-im.github.io/wis2-transition-guide/transition-guide/wis2-transition-guide-APPROVED.html

Annex A: FOSS evaluation rubric

Category	Indicator
Licence Compatibility	Licence is permissive (MIT, BSD, Apache 2.0)
	Compatible with NMHS mandates and data policies
	No restrictions on data sharing or commercial use
	Licence is OSI-approved
	Licence is clearly stated (no licence = reject)
Security	Clear security vulnerability policy/process
	Prompt patch releases (within days)
	No known unpatched CVEs
	Regular security checks (e.g., daily or automated)
Project Health	Public issue tracker
	Source code under version control (e.g. on GitHub, GitLab, Codeberg or other version control system)
	Active communication channels (forum, mailing list, chat)
	Wiki or documentation site
	Contribution guidelines or agreements
Community Health	Multiple active contributors
	Supported by major NMHSs, WMO Programmes, or recognized consortia
	Low bus factor (not reliant on one person)
	Code of conduct available
Openness & Governance	Transparent governance model
	Open contribution process
	Alignment with WMO standards (BUFR, GRIB2, NetCDF)
	Decision-making process documented
	Source code openly available (mandatory)
Technical Quality & Fit	High test coverage
	Continuous Integration (CI) in place
	Comprehensive documentation (user, developer, admin)
	Proven ability to handle NMHS-scale data volumes
	Validated in operational NMHS environments
	Clear release lifecycle and package management

Annex B: Examples of FOSS in the meteorological community

The following table provides examples of FOSS repositories maintained by WMO, its Members, and closely affiliated intergovernmental and research organizations, organized by WMO Regional Association (RA). It is not exhaustive; it illustrates the breadth and maturity of FOSS activity across the WMO community. All listed projects carry OSI-approved open source licences unless otherwise noted.

NOTE GitHub is the dominant platform for NMHS open source publishing. GitLab sees some use, both on gitlab.com and through self-hosted instances (e.g. EUMETSAT operates gitlab.eumetsat.int; GeoSphere Austria uses a self-hosted Forgejo instance at gitea.geosphere.at). The choice of platform is a matter of organizational preference and existing infrastructure; Members should evaluate options based on accessibility, governance, and their own policy requirements (see Source code hosting and collaboration).

Organization	Platform	Repository URL	Notable projects
WMO			
WMO Information Management (`wmo-im`)	GitHub	https://github.com/wmo-im	• wis2-gdc / wis2-gc / wis2-gb — WIS2 Global Discovery Catalogue, Global Cache, and Global Broker Reference Implementations (Apache-2.0) • pywcmp — Python tool for validating WMO Core Metadata Profile (WCMP) documents (MIT) • pywiscat — Python API for querying the WIS2 Global Discovery Catalogue (MIT) **NOTE** Whilst listed under the WMO, the listed software packages / RIs have been developed collaboratively with Canada and other Members

Organization	Platform	Repository URL	Notable projects
World Meteorological Organization (`World-Meteorological-Organization`)	GitHub	https://github.com/World-Meteorological-Organization	• wis2box — WIS2 in a box; reference implementation of a WMO WIS2 Node (Apache-2.0) • wis2box-ui / wis2box-webapp — Vue.js interfaces for wis2box (Apache-2.0) • synop2bufr — converts SYNOP FM-12 messages to BUFR (Apache-2.0) • csv2bufr — converts CSV observation data to BUFR (Apache-2.0) • pywis-pubsub — Python library for WIS2 MQTT data subscription and download (Apache-2.0) • cap-composer — WMO Common Alerting Protocol (CAP) composer tool (MIT) **NOTE** Whilst listed under the WMO, the wis2box software and other packages have been developed collaboratively with Canada and other Members
WMO Communities of Practice (`wmo-cop`)	GitHub	https://github.com/wmo-cop	• pyoscar — Python API client for WMO OSCAR/Surface station metadata registry (MIT)
Intergovernmental and multi-agency			
European Centre for Medium-Range Weather Forecasts (ECMWF)	GitHub	https://github.com/ecmwf	• eccodes — GRIB and BUFR encoding/decoding library; the de facto global standard (Apache-2.0) • cfgrib — Python interface mapping GRIB files to xarray/CF Convention (Apache-2.0) • cdsapi — Python API for the Copernicus Climate Data Store (Apache-2.0) • earthkit — Python tools for weather and climate data workflows (Apache-2.0) • magics — meteorological plotting and visualisation library (Apache-2.0) • Anemoi framework — collaborative ML weather forecasting framework co-developed with DMI, DWD, FMI, KNMI, MET Norway, Météo-France, MeteoSwiss, RMI Belgium, and AEMET (Apache-2.0)

Organization	Platform	Repository URL	Notable projects
Pytroll (multi-agency satellite data processing consortium) [1]	GitHub	https://github.com/pytroll	• satpy — Python package for Earth observation satellite data processing (GPL-3.0) • pyresample — geospatial image resampling for satellite data (LGPL-3.0) • pyorbital — orbital and astronomy computations for satellite passes (GPL-3.0)
RA I — Africa			
WMO Regional Office for Africa / NORCAP (`wmo-raf`)	GitHub	https://github.com/wmo-raf	Tools for African NMHS digital transformation (~74 repos, very active). Key projects: • climweb — open source CMS and public-facing website platform for NMHSs, including weather pages, forecasts and warnings (MIT) • adl — Automated Data Logger for weather observation data collection and processing (MIT) • cap-composer — CAP alerts composition and dissemination tool (MIT) • geomanager — geospatial data manager for raster and vector datasets (MIT)
Climsoft [2]	GitHub	https://github.com/climsoft	Widely-used open source Climate Data Management System for African NMHSs. Deployed operationally across East Africa and supported in 14+ SADC member states and 17 West African countries. • Climsoft — desktop application; operational in Kenya, Uganda, Ethiopia, Sudan, Djibouti (GPL-3.0) • climsoft-web — next-generation web rewrite (Climsoft v5); in active development (GPL-3.0)
RA II — Asia			

Organization	Platform	Repository URL	Notable projects
China Meteorological Administration — National Meteorological Centre (NMC)	GitHub	https://github.com/nmcdev	Python and R toolkit for operational meteorological development (19 repos, actively maintained). Key projects: • meteva — fast evaluation and validation library for weather forecast products (Apache-2.0) • nmc_met_io — reading and writing MICAPS files, satellite imagery, and radar data (GPL-3.0) • nmc_met_graphics — meteorological charting and colour table utilities (GPL-3.0) • metdig — meteorological diagnostic tools (GPL-3.0) • nmc_met_map — mapping utilities for meteorological applications (GPL-3.0)
RA III — South America			
Instituto Nacional de Pesquisas Espaciais (INPE, Brazil) [3]	GitHub	https://github.com/dissm-inpe	• goes2image — converts GOES/ABI satellite data to standard image formats; used for operational satellite imagery processing (MIT)
RA IV — North America, Central America and the Caribbean			
University Corporation for Atmospheric Research (UCAR) / NCAR [4]	GitHub	https://github.com/wrf-model	• WRF — Weather Research and Forecasting model; widely used mesoscale NWP system for research and operations (UCAR public domain licence — not OSI-certified but essentially unrestricted) • WPS — WRF Preprocessing System for terrain, land use and meteorological input data • WRFDA — data assimilation system for WRF
NOAA Environmental Modeling Center (EMC)	GitHub	https://github.com/NOAA-EMC	• WW3 — WAVEWATCH III ocean wave model (LGPL-3.0) • global-workflow — GFS/GEFS global forecast system workflow (LGPL-3.0) • GSI — Gridpoint Statistical Interpolation data assimilation system (LGPL-3.0) • NCEPLIBS-bufr — BUFR encoding/decoding library (LGPL-3.0)

Organization	Platform	Repository URL	Notable projects
NOAA Office of Water Prediction (OWP)	GitHub	https://github.com/NOAA-OWP	• ngen — Next Generation Water Modeling Engine (Apache-2.0) • inundation-mapping — flood inundation mapping software (Apache-2.0)
NOAA Geophysical Fluid Dynamics Laboratory (GFDL)	GitHub	https://github.com/NOAA-GFDL	• FMS — Flexible Modeling System for coupled climate models (Apache-2.0) • MOM6 — Modular Ocean Model version 6 (Apache-2.0)
Environment and Climate Change Canada — Meteorological Service of Canada (MSC)	GitHub	https://github.com/ECCC-MSC	• msc-pygeoapi — OGC API/pygeoapi deployment for MSC open data (MIT) • msc-animet — interactive weather animation tool for MSC Open Data (GPL-3.0) • libecbufr — BUFR encoding/decoding library (GPL-3.0) • msc-wis2node — MSC WIS2 Node implementation (GPL-3.0)
Environment and Climate Change Canada — Meteorological Research Division	GitHub	https://github.com/ECCC-ASTD-MRD	• gem — Global Environmental Multiscale (GEM) NWP model (LGPL-2.1) • librmn — NWP utility library (LGPL-2.1) • MIDAS-src — Modular and Integrated Data Assimilation System (LGPL-2.1)
NSF Unidata (USA)	GitHub	https://github.com/Unidata	• thredds — THREDDS Data Server for scientific data access via OPeNDAP, WMS and other protocols (BSD-3-Clause) • MetPy — Python toolkit for meteorological data reading, analysis and visualisation (BSD-3-Clause) • netcdf-c — NetCDF-C library (BSD-3-Clause) • LDM — Local Data Manager for real-time data distribution (BSD-2-Clause)
OpenDCS — Open Data Collection System [5]	GitHub	https://github.com/opendcs/opendcs	Java software for retrieving, decoding, processing, and storing hydrometeorological time series data from monitoring platforms, particularly via NOAA GOES satellite telemetry. (Apache-2.0)

Organization	Platform	Repository URL	Notable projects
National Meteorological Service of Belize (NMS-Belize)	GitHub	https://github.com/NMS-Belize	• surface — web-based surface observation management system for data entry, quality control and dissemination; actively maintained (GPL-3.0) • surface-installer — Python installer for the surface application (MIT)
RClimDex	GitHub	https://github.com/ECCC-CDAS/RClimDex	R package and GUI for computing the 27 ETCCDI temperature and precipitation extreme indices from daily station data, with built-in quality control. Widely used in WMO regional training workshops; predecessor to Climpact. (LGPL-2.1)
RA V — South-West Pacific			
Bureau of Meteorology — Radar Applications (`bom-radar`)	GitHub	https://github.com/bom-radar	• rapic — Rapic radar data protocol support library (Apache-2.0) • odim_h5 — OPERA Data Information Model (ODIM) HDF5 format support library (Apache-2.0) • lpats — LPATS lightning protocol support library (Apache-2.0)
CSIRO (Commonwealth Scientific and Industrial Research Organisation)	GitHub	https://github.com/csiro	• ccam-ccam — Conformal Cubic Atmospheric Model; variable-resolution global model for GCM downscaling and regional climate projections (GPL-3.0)
MetOcean Solutions (division of MetService, New Zealand)	GitHub	https://github.com/metocean	• wavespectra — wave spectra analysis tools (MIT) • moana-qc — quality control for ocean temperature observations (MIT)
Climpact [6]	GitHub	https://github.com/ARCCSS-extremes/climpact	R Shiny application and R package for calculating the WMO Expert Team on Sector-specific Climate Indices (ET-SCI) — a superset of the 27 ETCCDI indices extended to health, agriculture and water sectors. The WMO-endorsed reference tool for ET-SCI index calculation; used by NMHSs globally. (GPL-3.0)
RA VI — Europe			

Organization	Platform	Repository URL	Notable projects
Meteorologisk institutt (MET Norway)	GitHub	https://github.com/metno	• fimex — file interpolation, manipulation and extraction for meteorological grid data (GPL-2.0) • diana — operational meteorological visualisation and production software (GPL, version unconfirmed) • pyaerocom — Python tools for climate and air quality model evaluation (GPL-3.0) • bris-inference — operational ML weather model (Bris), based on Anemoi framework (Apache-2.0) • kvalobs — automated quality control of geophysical observations, co-developed with SMHI (GPL-2.0)
Finnish Meteorological Institute (FMI)	GitHub	https://github.com/fmidev	• smartmet-server — high-performance operational NWP data server; widely deployed internationally (MIT) • rack — weather radar data processing (MIT) • himan — weather parameter post-processing tool (MIT)
Danmarks Meteorologiske Institut (DMI)	GitHub	https://github.com/dmidk	• sunflow — solar nowcasting framework (MIT)
Swedish Meteorological and Hydrological Institute (SMHI)	GitHub	https://github.com/smhi	Components of the Diana visualisation ecosystem (shared with MET Norway); co-develops Kvalobs (see MET Norway entry).
Met Office (UK)	GitHub	https://github.com/MetOffice	• lfric_apps / lfric_core — LFRic, the next-generation atmospheric model (BSD-3-Clause) • CSET — toolkit for evaluation of weather and climate models (Apache-2.0)

Organization	Platform	Repository URL	Notable projects
Royal Netherlands Meteorological Institute (KNMI)	GitHub	https://github.com/KNMI	• adaguc-server — OGC WMS/WCS server for meteorological and climate data; widely deployed (Apache-2.0) • adaguc-viewer — companion JavaScript web viewer (Apache-2.0) • edr-pydantic — Pydantic models for OGC Environmental Data Retrieval API (MIT) • covjson-pydantic — Pydantic models for CoverageJSON (MIT)
Royal Netherlands Meteorological Institute (KNMI)	GitLab	https://gitlab.com/KNMI-OSS	• Climate Explorer — widely-used climate analysis web tool and data update scripts • Space weather tools — data retriever, HAPI server, and timeline viewer • KNMI public weather app and backend API
OpenGeoWeb consortium (KNMI, FMI, MET Norway)	GitLab	https://gitlab.com/opengeoweb	• opengeoweb — open source aviation meteorological workstation replacing legacy GeoWeb software; jointly developed by three NMHSs; 10,000+ commits, 200+ releases (Apache-2.0)
MeteoSwiss (Switzerland)	GitHub	https://github.com/MeteoSwiss	• ldcast — latent diffusion model for generative precipitation nowcasting (BSD-3-Clause) • pyrad — Python radar data processing library (BSD-3-Clause) • fast-barnes-py — fast Barnes interpolation for Python (BSD-3-Clause) • meteodata-lab — Python library for processing GRIB data with xarray (BSD-3-Clause)
Météo-France	GitHub	https://github.com/meteofrance	• mfai — Python package for operational ML workflows (Apache-2.0) • meteonet — toolbox and documentation for the MeteoNet open dataset (Apache-2.0) • py4cast — weather forecasting with deep learning (Apache-2.0)
Deutscher Wetterdienst (DWD)	GitHub	https://github.com/DeutscherWetterdienst	• regrid — tools to regrid ICON NWP data from icosahedral to regular grids (MIT) • downloader — Python CLI for downloading NWP GRIB2 data from the DWD Open Data server (MIT)

Organization	Platform	Repository URL	Notable projects
GeoSphere Austria (formerly ZAMG)	GitHub / Forgejo	https://github.com/Geosphere-Austria https://gitea.geosphere.at	Self-hosted Forgejo instance (gitea.geosphere.at) used as an internal scientific code and data management platform
National Technical University of Athens (NTUA) / openmeteo	GitHub	https://github.com/openmeteo/enhydris	• enhydris — Python/Django web application for storage, management and dissemination of hydrometeorological time series and station metadata; used by Greece's national hydrometeorological data bank (AGPL-3.0)
R-Instat [7]	GitHub	https://github.com/IDEMSInternational/R-Instat	Free, menu-driven statistical software powered by R, making climate analysis accessible to users without programming experience. Includes a dedicated climate menu for working with station data (normals, trends, wind roses, Climsoft integration). (GPL-3.0)

[1] Core contributors include EUMETSAT and several Nordic NMHSs

[2] Developed collaboratively by Zimbabwe Meteorological Department, Kenya Meteorological Department, ACMAD, and others, with support from WMO, UK Met Office, and IDEMS International

[3] INPE is Brazil's national space research institute and the primary source of operational satellite-derived meteorological data in the country.

[4] WRF is co-developed and maintained by NCAR, NOAA, the US Air Force, and numerous research and operational NMHSs worldwide

[5] Co-funded by USBR, USGS, and USACE; relevant to any NMHS using GOES DCS telemetry

[6] Developed at UNSW Sydney under the ARC Centre of Excellence for Climate Extremes; maintained by WMO CCl ET-SCI chair

[7] Developed by IDEMS International (UK) with the African Maths Initiative; primary user community is African NMHSs, with training documented in Kenya, Rwanda, Namibia, Zimbabwe, Uganda, Zambia, Niger, DRC, Botswana, and Djibouti